ÖBB

Kurityák Dániel

Az Ön feladata az Osztrák Szövetségi Vasutak (ÖBB) számára egy több aloldalból álló weboldal készítése.

Kezdje a szerveroldali kiszolgáló elkészítésével, majd foglalkozzon a kliensoldali alkalmazással!

Beállítások

1. Állítsa be a környezeti változókban az alábbiakat! Gondoskodjon róla, hogy a mintának szánt fájlban is módosítva legyenek!
   a. Az alkalmazás neve legyen *"ÖBB"*!
   b. Az adatbázis neve legyen `oebb`!
2. Másolja be a forrásban található `images` mappa tartalmát a `frontend/public/images` mappába!
3. Másolja be a forrásban található `openapi.yaml` fájlt a `swagger` mappába, kiscserélve ezzel az ott lévőt!
4. Indítsa el a konténereket, majd futtassa le az `oebb.sql` fájlt a phpMyAdmin felületén, az `oebb` adatbázisban!

Az adatok felépítése

A `hero_contents` tábla

A `hero_contents` tábla a főoldalon a hero szekcióban megjelenő tartalmat tárolja.

Mező neve	Típus	Megjegyzés
`id`	Egész [PK]	A tartalom azonosítója
`image_url`	Szöveg	A tartalomhoz tartozó képfájl neve
`title`	Szöveg	A tartalom címe
`description`	Szöveg	A tartalom leírása
`button_text`	Szöveg	A tartalomhoz tartozó gombon megjelenő szöveg

A `teasers` tábla

A `teasers` tábla a főoldalon lévő egyéb szekciókban megjelenő tartalmakat tárolja.

Mező neve	Típus	Megjegyzés
`id`	Egész [PK]	A tartalom azonosítója
`image_url`	Szöveg	A tartalomhoz tartozó képfájl neve
`title`	Szöveg	A tartalom címe
`description`	Szöveg	A tartalom leírása
`section`	Szöveg	Melyik szekcióhoz tartozik a tartalom
`order`	Egész	A szekción belül a tartalmak sorrendje

A `trains` tábla

A `teasers` tábla néhány nemzetközi vonat adatait tartalmazza.

Mező neve	Típus	Megjegyzés
`id`	Egész [PK]	A tartalom azonosítója
`train_number`	Szöveg	A vonat típusa és száma
`origin`	Szöveg	A kiinduló állomás neve
`destination`	Szöveg	Az célállomás neve
`departure_time`	Idő	A menetrendi induási idő
`arrival_time`	Idő	A menetrendi érkezési idő
`second_class_price`	Valós	Másodosztályú jegyár
`first_class_price`	Valós,	Elsőosztályú jegyár (ha elérhető)
`business_class`	Valós	Business osztály jegyár (ha elérhető)
`is_overnight`	Logikai	A vonat a következő nap érkezik-e

Backend programozás

Valósíts meg az alkalmazás szerveroldali kiszolgálót az alábbi és a `swagger.vm1.test` oldalon található leírás alapján!

5. Telepítse a Laravel API összetevőjét Artisan paranccsal!

Routing

6. Hozza létre az alábbi útvonalakat!

Method	URI	Name	Controller	Action
GET	`/api/hero-contents`	hero-contents.index	HeroContentController	index
GET	`/api/teasers`	teasers.index	TeaserController	index
GET	`/api/trains/{train}`	trains.show	TrainController	show
GET	`/api/timetable/stations/origins`	timetable.origins	TimetableController	origins
GET	`/api/timetable/stations/destinations`	timetable.destinations	TimetableController	destinations
GET	`/api/timetable/planner/{from}/{to}`	timetable.planner	TimetableController	planner

7. Az útvonalak paramétereit az alábbiak szerint validálja!
    a. A `{from}` és `{to}` paraméterek csak az alábbi karaktereket tartalmazhatja! Használjon reguláris kifejezést!
        - Bármilyen betű karakter (`\{}p{L}`)
        - Szóköz
        - Kötőjel
        - Pont
        - Perjel
    b. A `{train}` paraméter csak szám lehet!

Controller

`HeroContentController`

A `HeroContentController` kezeli a főoldalon a hero szekcióban megjelenő tartalmat.

8. Hozza létre a `HeroContentController` fájlt Artisan paranccsal!
9. Hozza létre az `index` action-t, amely egy `data` kulcsba csomagolva adja vissza az összes rekordot a `hero_contents` táblából!

`TeaserController`

A `TeaserController` kezeli a főoldalon lévő egyéb szekciókban megjelenő tartalmakat.

10. Hozza létre a `HeroContentController` fájlt Artisan paranccsal!
11. Hozza létre az `index` action-t, amely az alábbi leírás alapján adja vissza a rekordokat a `teasers` táblából!
    a. Amennyiben a query paraméterek között van `section`, úgy kizárólag az ahhoz a szekcióhoz tartozó tartalmakat kérdezze le, ügyelve a sorrend szerinti rendezésre!
    b. Amennyiben nincs ilyen query paramléter, úgy rendezés nélkül adja vissza az összes tartalmat!

`TrainController`

A `TrainController` kezeli néhány nemzetközi vonat adatait.

12. Hozza létre a `TrainController` fájlt Artisan paranccsal!
13. Hozza létre a `show` action-t, amely a paraméterül kapott azonosítójú vonat adatait adja vissza!
    a. Amennyiben nem létezik az adott azonosítójú vonat, úgy 404-es státusz kóddal térjen vissza!

`TimetableController`

A `TimetableController` kezeli a menetrendi útvonaltervezéseket.

13. Hozza létre a `TimetableController` fájlt Artisan paranccsal!
14. Hozza létre az `origins` action-t, amely a kiinduló állomások listáját adja vissza ABC sorrendben! Ügyeljen a Swagger-ben lévő mintára!
    a. A kiindulási állomások lekéréséhez használja a `trains` táblát!
    b. Minden kiindulási állomás csak egyszer jelenjen meg!
15. Hozza létre a `destinations` action-t, amely az érkezési állomások listáját adja vissza ABC sorrendben az alábbiak alapján! Ügyeljen a Swagger-ben lévő mintára!
    a. Az érkezési állomások lekéréséhez használja a `trains` táblát!
    b. Minden érkezési állomás csak egyszer jelenjen meg!
    c. Amennyiben a query paraméterek között van `origin`, úgy kizárólag azokat azokat az érkezési állomásokat adja vissza, ahova érkezik az `origin`-ben megadott állomásról vonat!
16. Hozza létra a `planner` action-t, amely két, a paraméterben megadott kiindulási (`from`) és érkezési (`to`) állomás között listázza ki a vonatokat az alábbi leírás alapján!
    a. Kizárólak a mintában szereplő mezőket adja vissza!
    b. Csak azokat a vonatokat adja vissza, ahol az `origin` és a `destination` mezők **részben** tartalmazzák a paraméterül megkapott értékeket!
    c. Rendezze a kapott adatokat az alábbiak szerint! A cél az, hogy a felhasználó a lehető legkorábban érkezzen, a lehető legkésőbb induljon, és a lehető legolcsóbban tudjon utazni!
        - Az éjszakán átnyúló vonatok szerint növekvő sorrendbe;
        - Az érkezési idő szerint növekvő sorrendbe;
        - Az indulási idő szerint csökkenő sorrendbe;
        - A másodosztályú jegyár szerint növekvő sorrendbe!
    d. Amennyiben a query paraméterek között van `time`, úgy a csak azokat a vonatokat jelenítse meg, amelyek a paraméterben megadottakkor vagy *később* **indulnak**!
    e. Amennyiben a query paraméterek között van `time` **és** `time-type`, **és** a `time-type` értéke *"arrival"*, úgy a csak azokat a vonatokat jelenítse meg, amelyek a paraméterben megadottakkor vagy *előbb* **érkeznek**!

Frontend programozás

A feladat során az oldalon nem minden gomb, link lesz fukcionális, azok a későbbiekben továbbfejleszthetők.

Az olal kialakítása során a formázásokhoz ahol lehet használjon TailwindCSS-t!

17. Állítsa át, hogy az oldal nyelve német legyen!
18. Valósítsa meg a kliensoldali útvonalválasztást az alábbiak szerint!

Útvonal neve	URI	Oldal címe
`home`	`/`	Startseite
`timetable.index`	`/timetable`	Fahrplan
`trains.show`	`/trains/:id`	Reisedetails

A főoldal (`home`)

A főoldalon a felhasználó megtekintheti a vasúttársaság éppen érvényes ajánlatait és híreit.

19. Állítsa be a `main.css`-ben, hogy a `primary` változó értéke `#e2002a`, a `primary-foreground` fehér legyen!

`Navbar` komponens

Minden oldalon, így a főoldalon is legfelül a navigációs sáv fog megjelnni. Szervezze ezt ki egy `Navbar` nevű komponensbe!

20. A forrásban kapott `navbar-template.html` fájlt alakítsa át egy `Navbar` Vue komponensé, majd végezze el az alábbiakat!
    a. Készítsen bele egy objektumokból álló tömnöt az alábbi táblázat alapján! A `page` kulcs értéke egy olyan objektum legyen, amelyet át tud adni egy `RouterLink` elemnek!

Tömb indexe	`title` kulcs értéke	`page` kulcs objektum melyik oldalra visz
0	Tickets & Ermäßigungen	`home`
1	Fahrplan	`timetable.index`
2	Reiseplanung & Services	`home`
3	Regionale Angebote	`home`

    a. A `div` elem alatt az előbb létrehozott tömböt felhasználva generáljon `RouterLink`-eket, amelyek a tömb elemében megadott szöveggel rendelkeznek és a megadott oldalra irányítanak át! Adja meg továbbá Tailwind segítségével, hogy szövege legyen középre igazítva, és flex-grow értéke 1 legyen!
    b. Kis méretű eszközökön a menüpontok egymás alatt jelennek meg. Viszont azt szeretnénk, hogy ezek a menüpontok csak szükség esetén legyenek láthatók. Szükség van ezért egy olyan állapotra, amelyben eltároljuk azt, hogy nyitva van-e a navbar. Hozzon létre egy állapotot `open` néven, amelynek alapértelmezett értéke legyen hamis!
    c. Számított állapotvel fogjuk eldönteni, hogy ha az állapot értéke igaz/hamis, akkor milyen osztályok legyenek alkalmazva a navbar-ra. Hozzon létre egy számított állapotot `openClass` néven, amely akkor, ha az `open` állapot értéke igaz, akkor a `max-h-300` string-et adja vissza, különben a `max-h-12 md:max-h-auto` string-et!
    d. Használja az előző feladatban létrehozott számított állapotot úgy, hogy az általa visszaadott osztály(ok) hozzá legyenek adva a meglévőkhöz a `nav` elemen!
    e. Szükség van egy, a főoldalra vivő menüpontra, illetve egy olyan gombra, amely kis méretű eszközökön nyitni és csukni tudja a menüt!
        - A `div` elemen belül helyezze el először a `BurgerIcon` komponenst, amelyre ha a felhasználó rányom, akkor az `open` állapot értéke változzon meg az azzal ellentétes értékre! Mivel közepes kijelzőktől felfelé már nincs szükség a gombra, így ilyen esetben rejtse azt el!
        - A `BurgerIcon` alatt helyezzen el egy `RouterLink`-et, amely irányítson át a főoldalra! Tartalma az `oebb-logo.svg` képfájl legyen *"ÖBB"* alternatív szöveggel!
21. Helyezze el közvetlen az oldal tetején az imént létrehozott `Navbar` komponenst!

Fő tartalmi rész

22. Helyezzen el a navbar alá egy `main` elemet, amely maximum szélessége legyen 340 egység, legyen középre igazítva és kapjon 12 egység felső belső margót!
23. Az oldalon nem fog megjelenni főcím, de a szövegfelolvasók miat mégis kell az oldalnak tartalmaznia. A `main.css` fájlban, az utolsó sorba illessze be az alábbi kódrészletet!

Az alábbi kód kiegészíti a `utilities` réteget egy `screen-reader-text` osztállyal, amely azokat az elemeket, amelyekre alkalmazva van, gyakorlatilag a szabad szem elől elrejti.

```
@layer utilities {
    .screen-reader-text {
        @apply absolute h-px w-px -m-px overflow-hidden p-0;
    }
}
```

24. Hozzon létre egy a `main` elembe egy főcmet, amely kapja meg az előbb létrehozott osztályt! Az elem szövege legyen *"ÖBB"*!
25. Készítsen egy állapotot `heroContents` néven, amelybe a hero szekcióban megjelenő elemeket fogjuk tárolni.
26. Készítsen egy `getHeroContents()` függvényt, amely lekéri a backend `/api/hero-contents` végpontjáról az összes hero-ba tartozó tartalmat, majd azt elmenti a `heroContents` állapotba!
27. Az `onMounted` függvénnyel állítsa be, hogy az oldal betöltődésekor a függvény automatikusan lefusson! Tesztelje a kérés sikerességét a böngésző fejlesztői eszköztárából!

Hero szekció

Az oldal hero része egy carousel lesz. Szervezze ezt ki egy `Hero` komponensbe!

28. Hozza létre a `Hero` komponenst, amely vegyen át a szülőelemetől egy `items` nevű tömböt, amelynek megadása kötelező (használja a `defineProps` függvényt)! A komponenst helyezze fel az oldalra, a főcím alá, a megfelelő értékeket átadva (`heroContents` állapot)!
29. A komponens stílusába másolja be az alábbi animációt, amely a carousel működéséért fog felelni!

```
@keyframes hero-carousel {
    0%,
    15% {
        transform: translate(0);
    }
    18%,
    32% {
        transform: translate(-100%);
    }
    34%,
    49% {
        transform: translate(-200%);
    }
    51%,
    67% {
        transform: translate(-300%);
    }
    69%,
    82% {
        transform: translate(-400%);
    }
    84%,
    98% {
        transform: translate(-500%);
    }
}
```

30. Helyezzen el a komponensbe egy `div` elemet, amely a vízszintes tengelyen rejtse el a túlcsorduló elemeket!
31. Az előbbi `div` elemen belül helyezzen el ismét egy `div` elemet, amely legyen fley elrendezésű, valamint kapja meg a `hero-carousel` keyframe-ű animációt, amely legyen 40 másodperc hosszú, és folyamatosan játszódjon le!

`HeroItem` komponens

32. A hero-ban megjelenő elemeknek hozzon létre egy külön komponenst `HeroItem` néven! A komponens a szülőelemtől a háttérképnek beállítandó kép fájlnevét kapja meg `image` néven, amely legyen szöveg típusú és kötelezően megadandó!

33. A forrásfájlok között található `hero-item-template.html` fájl tartalmazza az elem struktúráját. Másolja át és egészítse ki az alábbiakkal!
    a. A `style` attribútummal állítsa be, hogy a szülőelemtől megkapott kép jelenjen meg háttérképként, (többe között) kicserélve a *???* részt a megfelelőre!
    b. A `div` elem tartalmát a szülőelemben lehessen megadni! Használjon slot-ot!
34. Az így létrehozott `HeroItem` komponenst használja a `Hero` komponensen belül úgy, hogy a szülőelemtől kapott összes hero elemhez generáljon le egy `HeroItem` komponenst!
    a. Ügyeljen arra, hogy a ciklus által generált elemek a Vue-nak beazonsíthatóak legyenek (adja meg a `key` attribútumot)!
    b. A `HeroItem`-nek adja át a képfájl nevét!
35. A használt `HeroItem` komponens tartalmi részét töltse ki az alábbi elemekkel!
    a. A hero elem címét helyezze 3-as szintű címsorba, amely az alábbi formázásokat kapja!
        - szövegáryéka legyen 50%-ban áttetsző gray-800 színű, valamint közepes méretű
        - a szövege legyen középre igazítva, kivéve közepes méretű eszközöktől, ugyanis akkor balra legyen igazítva
        - szövege 2XL méretű legyen, kivéve közepes kijelzőtől, mert akkor már 4XL legyen
        - Legyen félkövér és kapjon 3 egység alsó margót
    b. A hero elem leírását helyezze egy bekezdésbe a 3-as szintű címsor alatt és formázza meg az alábbiakkal!
        - szövegáryéka legyen 50%-ban áttetsző gray-800 színű, valamint közepes méretű
        - a szövege legyen középre igazítva, kivéve közepes méretű eszközöktől, ugyanis akkor balra legyen igazítva
        - szövege SM méretű legyen, kivéve közepes kijelzőtől, mert akkor már MD legyen
        - Kapjon 2 egység alsó margót

`Button` komponens

36. Az oldalon előforduló gombok egységes formázása érdekében hozzon létre egy komponenst `Button` néven, amely a szülőelemtől kapja meg a `type` és `variant` nevű, opcionális stringeket!
    a. A `type` alapértelmezett értéke legyen `submit`!
    b. A `variant` alapértelmezett értéke legyen `primary`!
37. Hozzon létre egy `classes` objektumot, amely az alábbi két lehetséges `variant` értékhez (`primary` és `empty`):
    a. A `primary` esetén:
        - legyen `primary` háttérszínű és fehér szövegű
        - ha a felhasználó ráviszi a kurzort akkor fehér hátterű és `primary` betűszínű
        - körvonala akkor jelenjen meg, ha a felhasználó ráviszi az egeret, illletve legyen a színe `primary`!
    b. Az `empty` esetén legyen fehér háttrészínű és fekete betűszínű!
38. A komponensben legyen egy gomb, amely kapja meg a `type` értékét attribútumként, valamint a `px-4 py-3 cursor-pointer` osztályok mellett kapja meg az éppen akutális `vartiant` osztályait! A gombon belüli tartalmat a szülőelem adja meg!

A `Hero` komponens befejezése

39. Helyezze fel a `Button` komponenst a bekezdés alá, amelynek típusa legyen `button`, továbbá tegye ki a szülőelem szélességét, de ne lehessen szélesebb 95 egységnél, valamint alapértelmezetten legyen elrejtve, csak közepes eszközöktől jelenjen meg!
    a. A gomb tartalma két dolgo legyen:
        - a hero elemhez megjelenítendő gomb szövege
        - az `ArrowDownIcon` komponens, amely legyen sorközi elem és legyen úgy elforgatva, hogy lefelé helyett jobbra mutat a nyíl!

Teaser szekciók

40. A hero szekció alá helyezzen el egy `div` elemet, amely szélessége 95%-a a szülőelemének, de nem szélesebb 250 egységnél, valamint középre van igazítva.

`TeaserSection` komponens

41. Mivel több szekciónk is lesz a teaser-eknek, ezért hozzon létre egy `TeaserSection` komponenst! A komponens opcionálisan kapjon a szülőelemtől egy `title` nevű string-et!
42. A komponens tartalmába helyezzen el egy `div` elemet az alábbi formázásokkal:
    a. Legyen grid elrendezésű, kis méretű eszközökön 1 oszlopos, az elemek között 4 egység távolság legyen!
    b. Kapjon belül minden irányba, kívül csak függőlegesen 4 egység margót, de közepes méretű eszközöktől a melső margó mérete legyen 8 egység!
    c. Háttérszíne legyen fehér!
43. A `div` elem első gyermeke egy alcím legyen, ami csak akkor jelenjen meg, ha a `title` értéke meg van adva! Szintén ez az érték legyen az alcím szövege is!
    a. Az alcím legyen 3XL méretű, kapjon 5 pixel alsó margót és minden esetben tegye ki a rács egy teljes sorát!
44. Az alcím alá a szülőelemben megadott elemek kerüljenek be!

`Card` komponens

45. Az oldalon megjelenő kártyákat szeretnénk egységesíteni. Hozzon ehhez létre egy `Card` komponenst! A komponens az alábbi két értéket kapja a szülőtől!
    a. `image`, amely egy képfájl neve, amit meg kívánunk a kártyán jeleníteni. Szöveg típusú, nem kötelező megadni.
    b. `alt`, amely alternatív szöveg a képhez. Szintén szöveg típusú, nem kötelező megadni.
46. A komponens tartalmába helyezzen egy `div` elemet. Erre formázás nem szükséges.
47. A `div` elem első gyermeke egy kép legyen, amely a szülőtől kapott képet a szülőtől kapott alternatív szöveggel megjeleníti, abban az esetben, ha a képfájl neve meg lett adva! Lássa el még továbbá a `w-full` osztállyal!
48. A kép alá helyezzen egy `div` elemet, amely kapjon 4 egység belső margót és legyen fehér háttérszínű! Ebben az elemben a szülőelemben megadott elemek jelenjenek meg!

`TeaserCard` komponens

49. A teaser-ek esetében is többször lesz szükség az ezekre szabott kártyákra, ezért készítsen egy `Teaser` komponenst, amely paraméterül megkap kötelezően egy teaser objektumot!
50. A komponens tartalma legyen a `Card` komponens, amelynek képkétn adja át a teaser képét, valamint alternatív szövegként a teaser címét!
51. A `Card` komponensen belül helyezzen el egy 3-as szintű címsort, amely legyen LG méretű és kapjon 2 egység alsó margót. Szövege legyen a teaser címe!
52. A címsor alá helyezzen el egy bekezdést, amely szövege legyen a teaser leírása, valamint kapja meg az alábbi osztályokat!

    ```
    font-light text-sm text-ellipsis overflow-hidden whitespace-nowrap
    ```

Teaser-ek generálása

53. A teaser szekciók elemeit külön-külön kérje le! Hozza létre az alábbi állapotokat, valamint függvényeket, amelyek az adott backend címről lekérik a teaser szekciók elemeit és az állapotban eltárolják! Futtassa őket az oldal betöltődésekor!

Állapot neve	Függvény neve	Végpont címe
`newFromObbTeasers`	`getNewFromObbTeasers()`	`/api/teasers?section=new_from_obb`
`currentOffersTeasers`	`getCurrentOffersTeasers()`	`/api/teasers?section=current_offers`
`fromTheStatesTeasers`	`getFromTheStatesTeasers()`	`/api/teasers?section=from_the_states`

54. Az oldalon helyezzen a `div` elembe kettő `TeaserSection` komponenst! A tartományi ajánlatokra más megjelenítési mód lesz.
    a. Az első `TeaserSection` kapja meg a *"Neues von den ÖBB"* címet, és a `md:grid-cols-6` osztályt! A benne lévő elemeket a `newFromObbTeasers` állapot tartalmazza, az alábbi leírás alapján, a `TeaseCard`-ot generáljon belőlük!
        - Az első két elem szélesebb a rácsban, mint a többi. Ezért először csak az első két elemre hozzon létre egy `TeaserCard`-ot (`slice()` függvény), amelyek kapják meg a `md:col-span-3` osztályt!
        - A maradék elemekhez szintén hozzon létre egy-egy `TeaserCard` komponenst, de ebben az ő esetükben az osztályuk `md:col-span-2` legyen!

- Ne felejtse el megadni a komponensnek az adott teaser elem adatait, valamint a kulcsokat!

b. Az második `TeaserSection` kapja meg a *"Aktuelle Angebote"* címet, és a `md:grid-cols-3` osztályt! A benne lévő elemekt `currentOffersTeasers` állapot tartalmazza, generáljon az összeshez egy-egy a `TeaseCard`-ot!

- Ne felejtse el megadni a komponensnek az adott teaser elem adatait, valamint a kulcsokat!

A tartományoktól teaser szekció

55. Másolja fel a forrásfájlban elérhető `FromTheStatesSection`, `Stepper` és `StepperButton` komponenseket!
56. Egészítse ki a `Stepper` komponenst, úgy, hogy az első `StepperButton`-re klikkelve emit-elve legyen a `previous`, a másodikra kattintva a `next` esemény! Ne felejtse a `defineEmits`-et!
57. Használja a `FromTheStatesSection` komponenst arra, hogy az `items` attribútumának adja meg a `fromTheStatesTeasers` állapot tartalmát!

A menetrendi oldal

A menetrendi oldalon a felhasználó két állomás között tud útvonalat tervezni.

58. Készítsen egy állapotot `search` néven, amely a kereséshez tartozó adatokat egy objektumban fogja tartalmazni!
    a. A `from` és `to` kulcsok értékei legyenen üres stringek!
    b. A `date` legyen a mai dátum `éééé-hh-nn` formátumban (tipp: `toISOString()`)!
    c. A `time` értéke legyen a jelenlegi idő (tipp: `toLocaleTimeString( ... )`)!
59. Az oldal tetejére helyezze el a `Navbar` komponenst!
60. A `Navbar` komponens alá helyezzen el egy `main` elemet! Az elemnek állítsa be, hogy háttere a `timetable-background.png` legyen, amely csak az X tengelyen ismétlődjön, valamint kapjon 12 egység felső belső margót!

`PlannerForm`

61. A tervezéshez használandó űrlap több oldalon is alkalmazható lehetne. Hozzon létre egy `PlannerForm` komponenst, amely kapjon `v-model`-t a szülőelemtől! Ez egy objektum lesz, amely a szűrési feltételeket tartalmazza!
62. Tárolja el a mai dátumot úgy, hogy a `toISOString` függvény eredményének első 10 karakterét táolja el! (pl. eredmény: `2026-10-09`)
63. Kérje le a lehetséges indulási állomásokat a backend `/api/timetable/stations/origins` végpontjáról és tárolja el őket egy állapotban! A weboldal betöltődésére kérje le ezeket az adatokat!
64. Kérje le a lehetséges érkezési állomásokat a `/api/timetable/stations/destinations` végpontról! A függvény kapjon egy `origin` paramétert, amely ha meg van adva, akkor az azonos nevű query paraméternek az URL-ben legyen megadva mint érték! Az így kapott eredményeket szintán tárolja el egy állapotba!

A komponens fogja használni a saját `Button` komponenst, valamint szükséges egy új létrehozása `Input` néven!

`Input` komponens

65. Készítsen egy `Input` komponenst, amely kapjon egy `v-model`-t!
66. A komponensen belül legyen egy beviteli mező, amely kapja meg ezt a model-t, valamint az alábbi formázásokat:
    a. Betűmérete legyen LG és kapjon 4 egység belső margót!
    b. Kapjon `gray-600` színú szegélyt, valamint a sarkai legyenek lekerekítve!
    c. Abban az esetben, ha fókuszba kerül, kapjon `primary` színű outline-t!

A `PlannerForm` folytatása

67. Másolja be a forrásokból a `planner-form-template.html` tartalmát! Cserélje ki, hogy a natív beviteli mezők és gombok helyett a sajt komponensek szerepeljenek!
68. A model-ként kapott értékeket állítsa be `v-model`-ként az alábbi `Input` komponenseknek!

name	v-model	Validáció	Autocomplete
from	model.from	kötelező	kikapcsolva
to	model.to	kötelező	kikapcsolva
date	model.date	kötelező, legalább a mai nap	kikapcsolva
time	model.time	kötelező	kikapcsolva

69. A `from` input esetében, abban az esetben, hogy ha a `focusout` állapot megtörténik, úgy futtassa le a `getDestinations()` függvényt, amely kapja meg argumentumként az inputba beírt értéket!
70. Készítsen két számított állapotot, az egyiket `filteredOrigins`, a másikat `filteredDestinations` néven! Működésük megegyező, a különbség csupán annyi, hogy melyik adatokkal dolgoznak!
    a. A számított állapotok azokat az értékeket adják vissza az `origins` vagy `destinations` állapotból, amelyek tartalmazzák a hozzájuk tartozó keresőmezőek értékét bárhol a stringben! Ügyeljen rá, hogy a kis- és nagybetűk ne számítsanak!
71. Az így szűrt adatokból építse fel a `from` és `to` beviteli mezők alatt található listákat, amelyek javaslatokat fognak tartalmazni a létező állomásokkal!
    a. A felsorolások csak akkor jelenjenek meg, ha a hozzá tartozó számított állapotban lévő tömbnek vannak elemei!
    b. Generáljon egy-egy listaelemet az összes tömb elemhez! Ügyeljen a kulcsokra!
    c. Ha a listaelemen a `mousedown` esemény megtörténik, úgy állítsa be az adott keresési mezőt a listaelemhez tartozó értékre!

A menetrendi oldal folytatása

72. Az űrlap elküldését külön függvény fogja kezelni. Ez a függvény meghívja majd a `getJourneys()` függvényt. Készítse ezt el az alábbi leírás alapján!
    a. A függvény kérje le a `search` állapotot felhasználva a kiindulási állomás, az érkezési állomás, illetve a query paraméterben megadva az idő alapján a `/api/timetable/planner/{from}/{to}` végpontról.
    b. A kiindulási és érkezési állomások esetében a szövegben előfordulő összes `/` karaktert cserélje le `%2F`-re a kérés elküldésénél!
    c. A kapott eredményeket tárolja el agy `journeys` állapotban!
73. Vegyen fel további két állapotot!
    a. A `searched` azt fogja meghatározni, hogy keresett-e az oldal megnyitása óta a felhasználó. Legyen alapértelmezetten hamis!
    b. Az `isLoadingTimetable` azt határozza meg, hogy éppen van-e folyamatban betöltés a szerverről. Legyen szintén alapértelmezetten hamis!
74. Helyezze el a `PlannerForm`-ot a `main`-en belül, kapcsolja rá `v-model`-el a `search` állpotot!
75. Az űrlap elküldés eseményére alkalmazza a `prevent` modifier-t, illetve helyezzen rá egy eseménykezelőt `handleSubmit` néven!
76. Készítse el az előző feladatban behivatkozott `handleSubmit()` függvényt!
    a. A `searched` és `isLoadingTimetable` állapotok értékei legyenek igazak!
    b. Törölje a `journeys` állapot elemeit!
    c. Kérje le a `getJourneys()` függvénnyel a két pont közötti vonatokat!
    d. Állítsa át az `isLoadingTimetable` állapot értékét hamisra!
77. A `PlannerForm` alatt helyezzen el egy `div` elemet az alábbi osztályokkal: `max-w-250 mx-auto grid md:grid-cols-2 my-12 w`
78. A `div` elembe jelenítse meg a (forrásban megtalálható) `UserAccountCard` komponenst és `vehicles.png` képet **együttesen** akor, ha még nem volt keresés (a `searched` állapot értéke hamis).
79. Ha már volt keresés, akkor helyezzen bele egy `div` elemet `grid md:grid-cols-[1fr_auto] gap-4 w-full col-span-full` osztályokkal!
    a. Amennyiben az adatok még töltődnek, úgy jelenítse meg a `LoadingIcon` komponenst, amely kapja meg a `self-center` osztályt!
    b. Amennyiben legalább egy járat adata megtalálható a `journeys` állapotban, úgy helyezze el az alábbiakat:
        - Egy alcímet *"Verbindungen"* szöveggel és `screen-reader-text` osztállyal (a szövegfelolvasók érdekében)
        - Egy (a források között megtalálható) `TrainCard` komponenst, amelyből készítsen annyit, ahány vonat adatai visszatértek. Adja át a komponensnek a vonatok adatait a `train` attribútumon kersztül!

a. Amennyiben a `journeys` hosszúsága 0, úgy jelenjen meg (a források között szintén megtalálható) `Alert` komponens a következő attribútumokkal!

- `title` értéke:

  `Keine Reise gefunden`

- `text` értéke:

  `Bitte überprüfen Sie Ihren Abfahrts- und Ankunftsort, sowie das gewählte Datum und die angegebenen Reisenden.`

80. Ezen az oldalon a body háttérszíne legyen fehér!

A vonat információs oldal

`TickerCard` komponens

81. Módosítsa úgy a forrásban található `TickerCard` komponenst, hogy a benne lévő gonbra kattintáskot emit-elje a `selected` eseményt!

Az oldal beállításai

82. Helyezze el felülre a `NavBar` komponenst!
83. A `Navbar` alá helyezzen egy `main` elemet az alábbi formázásokkal!

    `max-w-340 mx-auto pt-12 w-[95%] bg-white min-h-[calc(100dvh-3rem)]`

84. Készítsen egy állapotot `train` néven, amelybe a lekért járat adatai fognak tartozni!
85. Készítsen egy függvényt `getTrain` néven, amely lekéri a `/api/trains/{id}` végpontról egy járat adatát, majd az eredményt eltárolja a `train` állapotban! Ne felejde azt, hogy mire van szüksége a route paramétereinek kinyeréséhez!
86. Vegyen fel egy állapotot `selectedTicket` néven, amely a kijelölt jegyet fogja eltárolni! Alapértelmezett értéke legyen -1 (azaz nincs jegy kiválasztva)
87. Az `isLoading` állapot adja meg, hogy az oldal éppen betölt-e! Az alapértelmezett értéke legyen az igaz!
88. Az oldal betöltődésekor töltse be a járat adatait, majd állítsa az `isLoading` értékét hamisra!
89. A `main` elemem belül az `LoadingIcon` jelenjen meg egészen addig, amíg az oldal tölt! Kapja meg a `self-center` osztályt!
90. Amennyiben nem tölt az oldal, úgy az alábbiak legyenek a `main` **közvetlen gyermekei**!

    a. Főcím, amely tratalma legyen *"Reisedetails"*, valamint legyen az alábbi formázású!

    - 3 XL betűméretű, félkövér
    - Felül 12 egység, alul 4 egység külső margója legyen!

    b. Jelenjen meg alatta a források között megtalálható `TrainInfo` komponens, amely `train` attribútuma legyen az aktuális járat adatai!

    c. Alatta egy `div` elem legyen, amely legyen ellátva az alábbi osztályokkal! `grid md:grid-cols-[repeat(auto-fit,minmax(250px,1fr))] gap-4 justify-between mt-8`
    A következő feladatokat ennek írja mint gyermekelem!

91. Helyezzen el egy alcímet *"Ticketangebot"* szöveggel és `screen-reader-text`, hogy az oldalon ne jelenjen meg, de szövegfeleolvasók lássák!
92. Helyezzen egy 3 darab `TicketCard` komponenst!

    a. Az elemeknek a `title` attribútumban megadott értékük jelenik meg mint cím.

    b. A `price` attribútumban megadott érték fog megjelenni mint ár.

    c. Az előbb létrehozott `selected` esemény a jegy kiválasztásakor fut le.

    d. Az alábbi táblázat alapján helyezze fel a `TicketCard` elemeket! Az elem csak akkor jelenjen meg, ha a `price`-ban megadott érték nem null/undefined! A táblázat *"Körvonal feltétele"* azt határozza meg, hogy mikor kerüljön rá a `outline-2` osztály (jelezve a kijelölést)!

`title`	`price`	**Körvonal feltétele**	`selected` **eseménykor**
2. Klasse	`train.second_class_price`	`selectedTicket == 0`	`selectedTicket = 0`
1. Klasse	`train.first_class_price`	`selectedTicket == 1`	`selectedTicket = 1`
Business Class	`train.business_class_price`	`selectedTicket == 2`	`selectedTicket = 2`

e. Ezeken kívül helyezzen fel még egy `TrainCard` komponens, ami csak akkor jelenjen meg, ha a vonat száma az alábbiak egyikével kezdődik!

`RJ, RJX, NJ, EN, IC, IR, EC, ICE, D`

- Kapja meg a `reservationOnly` attribútumot!

`title`	`price`	**Körvonal feltétele**	`selected` **eseménykor**
Nur Sitzplatz reservieren	`2`	`selectedTicket == 3`	`selectedTicket = 3`